INDIAN INSTITUTE OF INFORMATION TECHNOLOGY
BHOPAL (M.P.)
Department of Computer Science and Engineering

B.Tech Major Project Report
(Phase – II)

FEDERATED MULTIMODAL GRAPH LEARNING FRAMEWORK FOR IN-HOSPITAL AND ICU MORTALITY PREDICTION

Submitted in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in Computer Science and Engineering

Submitted by:

Dhruv Savaliya
Scholar No: 21U02035

Menil Patel
Scholar No: 21U02052

Under the Supervision of:

Project Supervisor
Assistant Professor
Department of CSE
IIIT Bhopal (M.P.)

B.Tech CSE – Final Year (Semester VIII)
Academic Year: 2025 – 2026
October 2026

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY, BHOPAL (M.P.)

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the work embodied in this project report entitled "FEDERATED MULTIMODAL GRAPH LEARNING FRAMEWORK FOR IN-HOSPITAL AND ICU MORTALITY PREDICTION" has been satisfactorily completed by the following students of the Department of Computer Science and Engineering, Indian Institute of Information Technology, Bhopal:

1. **Dhruv Savaliya (Scholar No. 21U02035)**
2. **Menil Patel (Scholar No. 21U02052)**

It is a bonafide piece of work, carried out under our guidance for the partial fulfillment of the requirements of the degree of Bachelor of Technology in Computer Science and Engineering during the academic year 2025–2026. The work presented in this report has not been submitted elsewhere for the award of any other degree or diploma.

Date: _____

Place: Bhopal (M.P.)

_____	_____
Project Supervisor	**Head of Department**
Department of CSE	**Department of CSE**
IIIT Bhopal (M.P.)	**IIIT Bhopal (M.P.)**

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY, BHOPAL (M.P.)

Department of Computer Science and Engineering

DECLARATION

We hereby declare that the major project report entitled "FEDERATED MULTIMODAL GRAPH LEARNING FRAMEWORK FOR IN-HOSPITAL AND ICU MORTALITY PREDICTION" submitted herein is a genuine record of research and engineering work carried out by us in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering from the Indian Institute of Information Technology, Bhopal.

The work presented in this project report has not been submitted in part or in whole for the award of any degree or diploma in any other institute or university. We further declare that all data, source code, experimental results, figures, and analyses presented in this document are authentic and original. All external references have been duly cited in the IEEE format.

In case of any discrepancy or factual inaccuracy found in this report, we shall bear full responsibility.

Dhruv Savaliya
Scholar No: 21U02035
B.Tech CSE, Final Year

Menil Patel
Scholar No: 21U02052
B.Tech CSE, Final Year

ACKNOWLEDGEMENT

We would like to express our deepest and most sincere gratitude to everyone who supported, encouraged, and guided us throughout the successful conceptualization, implementation, and completion of our B.Tech major project titled "Federated Multimodal Graph Learning Framework for In-Hospital and ICU Mortality Prediction."

First and foremost, we extend our heartfelt gratitude to the esteemed Director, Indian Institute of Information Technology Bhopal, for providing state-of-the-art computational infrastructure, laboratory facilities, and an inspiring academic environment that made rigorous machine learning research possible.

We are profoundly grateful to our Project Supervisor for invaluable guidance, patient mentoring, perceptive critique, and continuous encouragement throughout the lifecycle of this research. Their deep domain expertise in artificial intelligence, graph representation learning, and federated systems provided us with the direction required to design and evaluate a federated predictive framework.

We also convey our gratitude to the Head of the Department and Department Coordinator, Department of Computer Science and Engineering, for administrative support, curriculum coordination, and constructive evaluations during project progress reviews.

Our sincere thanks are extended to all faculty members and technical staff of the CSE Department for theoretical guidance and technical support. We are equally grateful to our fellow classmates and peers for stimulating technical discussions and constructive feedback.

Finally, we express our profound gratitude to our parents and family members for their unconditional love, moral support, and endless patience, which served as the cornerstone of our perseverance throughout this challenging endeavor.

Dhruv Savaliya (21U02035)
Menil Patel (21U02052)

ABSTRACT

In modern intensive care units (ICUs), early and accurate risk stratification for life-threatening outcomes such as in-hospital and ICU mortality is vital for timely clinical intervention and resource allocation. Although machine learning models trained on Electronic Health Records (EHRs) have shown significant promise, their real-world deployment faces two major structural barriers: (1) privacy regulations (e.g., HIPAA, GDPR, DPDPA) that restrict centralizing sensitive protected health information across hospital networks, and (2) the failure of traditional models to exploit relational inductive biases between clinically similar patients who share acute physiological phenotypes.

To study this tension between clinical privacy and predictive utility, this project implements a Federated Multimodal Graph Learning Framework and evaluates it in a single-machine federated simulation on the multi-center eICU Collaborative Research Database (207 hospitals, 200,826 ICU encounters). Hospitals are split into 145 training, 31 validation and 31 test hospitals. The task is early prediction: each model sees only the first 24 hours of an ICU stay and predicts in-hospital and ICU mortality for patients still in the ICU at 24 hours (132,878 encounters). For each patient, the record is decomposed into two complementary modalities: (i) static tabular variables (demographics, admission diagnoses, baseline organ scores) encoded via a multi-layer perceptron into a 64-dimensional latent embedding, and (ii) time-series observations in 4-hour bins (laboratory tests, infusion rates, medications and treatments) modeled via a 2-layer Long Short-Term Memory (LSTM) network with packed sequence handling into a 64-dimensional temporal embedding. A multimodal fusion layer projects the concatenated representations into a 128-dimensional unified patient space.

Within each hospital client, a patient similarity graph is built for every training batch via k-nearest neighbors (k=5) using cosine similarity, so edges only connect patients of the same hospital. A Graph Neural Network (PatientGNN) leveraging GraphSAGE, Graph Attention Networks (GAT), or Graph Convolutional Networks (GCN) propagates neighborhood clinical signals, capturing phenotypic correlations among acute patients. Attached dual multi-task prediction heads simultaneously predict In-Hospital Mortality and ICU Mortality using a positive-class weighted multi-task loss function. A central server aggregates local model weights via Federated Averaging (FedAvg), weighted by the number of examples each hospital trained on, so raw patient rows are not sent to the server in the simulation.

On the 31 held-out test hospitals (18,014 patients), averaged over three training seeds, the proposed FedAvg + GNN model reaches an in-hospital mortality AUROC of 0.819 ± 0.007 (AUPRC 0.400 ± 0.009) and an ICU mortality AUROC of 0.853 ± 0.005. It is consistently better than local-only training on a single hospital (AUROC 0.769, 0.660 and 0.583 for hospitals of 1,562, 458 and 87 patients), and the gap grows as the single hospital gets smaller. Removing the graph (No-GNN ablation) lowers the in-hospital mortality AUROC slightly to 0.806 ± 0.024, so

the graph gives a small but consistent gain on this task rather than a large one. These results show that a simple federated multimodal graph model can learn useful early mortality risk from decentralized ICU data; they come from a single-machine simulation and are not a clinical validation.

Keywords: *Federated Learning, Graph Neural Networks, Multimodal Deep Learning, eICU Dataset, ICU Mortality Prediction, Mortality Risk Stratification, Federated Simulation, Patient Similarity Networks.*

TABLE OF CONTENTS

Section	Title	Page
	Certificate	2
	Declaration	3
	Acknowledgement	4
	Abstract	5
	Table of Contents	7
	List of Figures	10
	List of Tables	10
1	**Introduction**	12
1.1	Clinical Background & Motivation	12
1.2	The Privacy-Utility Conundrum in Healthcare	12
1.3	Challenges in Multimodal Critical Care Modeling	13
1.4	Proposed Federated Graph Learning Paradigm	13
1.5	Organization of the Report	13
2	**Literature Review**	15
2.1	Clinical Risk Stratification Systems	15
2.2	Deep Learning for Electronic Health Records	15
2.3	Multimodal Representation Learning in Medicine	16
2.4	Graph Neural Networks & Patient Similarity Networks	16
2.5	Federated Learning in Distributed Healthcare	17
2.6	Critical Analysis and Research Gaps	18

3	**Problem Definition and Objectives**	19
3.1	Formal Problem Formulation	19
3.2	Research Objectives	19
3.3	Technical Scope & Boundary Conditions	20
4	**Proposed Methodology and Work Description**	21
4.1	System Architectural Overview	21
4.2	Multimodal Data Ingestion & Canonical Alignment	22
4.3	Modality-Specific Encoders	22
4.4	Multimodal Non-Linear Fusion Layer	23
4.5	Intra-Hospital Patient Graph Construction	23
4.6	Graph Representation Learning (PatientGNN)	24
4.7	Multi-Task Clinical Prediction Heads	24
4.8	Joint Multi-Task Objective with Imbalance Weighting	24
5	**Proposed Algorithms**	26
5.1	Client-Side Local Multimodal Graph Training	26
5.2	Server-Side Federated Averaging Coordination	26
6	**Flowchart, Block Diagram, and DFD**	28
6.1	System Architectural Block Diagram	28
6.2	Data Flow Diagram (DFD)	28
6.3	Graph Construction Flowchart	29

7	**Implementation Details & Technology Stack**	30
7.1	Hardware & Software Environments	30
7.2	Dataset Specification (eICU Database)	30
7.3	Data Pipeline Implementation	31
7.4	Deep Learning & GNN Modules	31
7.5	Federated Training Engine	32
8	**Result Discussion and Comparative Analysis**	34
8.1	Experimental Setup & Evaluation Protocol	34
8.2	Clinical Evaluation Metrics	34
8.3	Federated Convergence Dynamics	34
8.4	Comparative Baseline Evaluation	35
8.5	Patient Graph Visualization	37
9	**Conclusion and Future Scope**	39
9.1	Summary of Contributions	39
9.2	Limitations of the Current Study	39
9.3	Future Research Directions	40
	References (IEEE Format)	41
	List of Publications	43
	Plagiarism & Authenticity Report	44

LIST OF FIGURES

Figure No.	Description	Page No.
Figure 1	Federated Multimodal Graph Learning End-to-End System Architecture	28
Figure 2	Data Flow Diagram (Level 0 Context and Level 1 Functional Pipeline)	29
Figure 3	Hospital Patient Population Scale and Distribution across 207 eICU Sites	31
Figure 4	Validation Loss and Mortality AUROC per Round (FedAvg + GNN vs No-GNN, mean ± SD over 3 seeds)	35
Figure 5	Test-Hospital Performance: FedAvg + GNN vs No-GNN vs Local-Only	37
Figure 6	Patient Similarity Graph Built by the Trained Model on Test Hospital 120	38

LIST OF TABLES

Table No.	Description	Page No.
Table 1	Multimodal Feature Modalities in Critical Care Medicine	16
Table 2	Comparison of Contemporary Machine Learning Healthcare Frameworks	17
Table 3	Canonical Feature Dimensions and Modality Specifications	21

Table No.	Description	Page No.
Table 4	Hyperparameter Configuration for Federated Patient Graph Model	32
Table 5	Validation Convergence of the Proposed FedAvg + GNN Model (mean of 3 seeds)	35
Table 6	Test-Hospital Results (mean ± SD over 3 seeds)	36

CHAPTER 1
INTRODUCTION

1.1 Clinical Background & Motivation

The Intensive Care Unit (ICU) represents the most technologically intensive and data-dense environment in modern healthcare. Critically ill patients admitted to ICUs suffer from severe, rapidly progressing pathologies such as septic shock, multi-organ failure, and acute respiratory distress syndrome (ARDS). Among these conditions, sepsis remains a leading cause of in-hospital mortality and healthcare expenditure worldwide, accounting for over 11 million global deaths annually [4]. In critical care settings, patient deterioration often unfolds across hours or even minutes. Traditional clinical monitoring relies heavily on episodic physician examinations and heuristic scoring systems such as the Acute Physiology and Chronic Health Evaluation (APACHE) [1] and Sequential Organ Failure Assessment (SOFA) [3]. While clinically grounded, these systems are fundamentally static, labor-intensive, and inadequate for dynamic, real-time risk stratification.

The ubiquity of Electronic Health Record (EHR) systems in modern hospitals has made continuous physiological data available at unprecedented scale. Every hour, bedside monitors record physiological waveforms, clinical laboratories produce biochemistry panels, and clinicians administer titrated vasopressors and fluid boluses. Developing predictive artificial intelligence (AI) systems capable of using these multi-stream inputs to estimate the risk of death early in the ICU stay, while there is still time to act, is therefore of clear clinical importance [5].

1.2 The Privacy-Utility Conundrum in Healthcare

Despite the availability of powerful machine learning architectures, real-world deployment across multi-hospital health systems faces a critical bottleneck: data governance and patient confidentiality. Electronic health records contain highly sensitive Protected Health Information (PHI). Legal frameworks such as the Health Insurance Portability and Accountability Act (HIPAA) in the United States, the General Data Protection Regulation (GDPR) in the European Union, and the Digital Personal Data Protection Act (DPDPA) in India strictly govern the transmission and centralized pooling of healthcare records across jurisdictional and organizational boundaries.

Historically, training deep learning models required centralizing raw patient records into a single data warehouse. For independent hospitals, this centralized model introduces insurmountable legal liabilities, regulatory compliance hazards, and commercial vulnerabilities. Conversely, training models on data from a single hospital in isolation causes severe algorithmic bias and over-fitting, as individual sites exhibit idiosyncratic demographics, local clinical workflows, and limited statistical samples. This tension between institutional data compartmentalization and cross-site statistical generalizability is known as the healthcare privacy-utility conundrum [22].

1.3 Challenges in Multimodal Critical Care Modeling

Modeling intensive care EHR datasets introduces three distinct technical challenges that demand specialized engineering solutions:

Challenge 1 — Heterogeneous Modality Representation: ICU patient records are inherently multimodal. They encompass static tabular demographics (age, biological sex, admission type, chronic comorbidities) and dynamic time-series (laboratory results, intravenous infusions, medications and treatments). Static variables capture baseline physiological reserve, whereas temporal streams capture acute deterioration trajectories. Fusing these fundamentally disparate temporal granularities requires specialized modality encoders [11].

Challenge 2 — Inter-Patient Clinical Relatedness: Conventional machine learning models treat each patient admission as an independent, identically distributed (I.I.D.) vector in Euclidean space. However, clinical pathology is profoundly relational. Patients exhibiting similar hemodynamic phenotypes (e.g., refractory hypotension coupled with elevated serum lactate) share underlying pathophysiological risk trajectories. Ignoring patient similarity networks discards valuable relational inductive biases [15].

Challenge 3 — Class Imbalance & Non-I.I.D. Hospitals: death is a low-frequency, high-stakes outcome: in the eICU data used here, 9.0% of ICU encounters end in in-hospital death and 5.4% in ICU death. Furthermore, patient volumes across community hospitals and tertiary academic medical centers vary by orders of magnitude (from 3 to 7,059 encounters per hospital in the eICU database), creating strong statistical heterogeneity [25].

1.4 Proposed Federated Graph Learning Paradigm

To overcome these challenges, this project presents an end-to-end Federated Multimodal Graph Learning Framework for early In-Hospital and ICU Mortality prediction. In the federated design, raw patient records stay with each hospital; in this project the hospitals are simulated as separate clients inside one program on one machine. Each hospital client independently executes feature extraction, modality encoding (Tabular MLP + Time-series LSTM), and multimodal fusion. Within each hospital, a patient graph is constructed for every mini-batch using k-nearest neighbors (k-NN) over unified patient embeddings. A Graph Neural Network (PatientGNN) then passes information between similar patients. Only model weights are sent to the central server for Federated Averaging (FedAvg) aggregation [20]. The project combines multimodal representation learning, graph-based patient similarity and federated optimization in one prototype, and measures on held-out hospitals whether each part helps.

1.5 Organization of the Report

The remainder of this report is organized as follows:

• Chapter 2 conducts a comprehensive literature review of clinical risk scores, EHR deep learning, multimodal fusion techniques, graph neural networks, and federated learning systems.
• Chapter 3 defines the formal problem statement, mathematical notation, and project objectives.
• Chapter 4 elaborates the proposed system methodology and component architectures.
• Chapter 5 formalizes the client-side and server-side algorithms.
• Chapter 6 presents architectural block diagrams and data flow diagrams.
• Chapter 7 details the software implementation and experimental environment.
• Chapter 8 presents empirical results, convergence curves, and comparative benchmarks.
• Chapter 9 concludes the study and outlines future research avenues.

CHAPTER 2
LITERATURE REVIEW

2.1 Clinical Risk Stratification Systems in Intensive Care

Risk stratification in critical care has traditionally relied on physiological scoring indices developed through multi-center logistic regression. The Acute Physiology and Chronic Health Evaluation (APACHE II, III, IV) developed by Knaus et al. [1] computes a composite integer score based on worst-observed vital signs and laboratory parameters during the initial 24 hours of ICU admission. Similarly, the Simplified Acute Physiology Score (SAPS II) [2] and the Sequential Organ Failure Assessment (SOFA) [3] quantify acute morbidity across cardiovascular, respiratory, neurological, renal, hepatic, and hematological organ systems.

In the context of sepsis, the Third International Consensus Definitions for Sepsis and Septic Shock (Sepsis-3) [4] operationalized sepsis as life-threatening organ dysfunction caused by a dysregulated host response to infection, clinically identified by an acute increase in SOFA score of 2 or more points. Although universally utilized for clinical benchmarking, these scoring systems suffer from profound structural limitations: (i) they rely on coarse physiological discretizations, (ii) they fail to model complex non-linear interactions across organ systems, (iii) they discard the rich temporal dynamics of continuous vital sign signals, and (iv) they cannot adapt dynamically to evolving clinical trajectories.

2.2 Deep Learning for Electronic Health Records (EHR)

The advent of deep neural networks enabled direct learning of hierarchical feature representations from high-dimensional EHR data [5]. Early breakthroughs utilized Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks to model irregular temporal sequences. Choi et al. [6] introduced Doctor AI, applying recurrent networks to predict differential diagnoses and medication orders over longitudinal patient visits. Lipton et al. [7] demonstrated that LSTMs directly handling missing values and measurement frequencies outperformed classical clinical baselines in predicting 128 diagnostic categories from pediatric intensive care time-series.

Subsequent research explored Transformer-based self-attention mechanisms. Vaswani et al. [8] revolutionized sequence modeling, leading to clinical adaptations such as Med-BERT [9] and BEHRT [10], which pre-train bidirectional attention representations over structured medical codes. While attention mechanisms excel at capturing long-range dependencies, standard recurrent and transformer models typically process each patient in isolation, ignoring the non-Euclidean similarity relationships existing across patient populations admitted with comparable disease etiologies.

2.3 Multimodal Representation Learning in Medicine

Clinical medicine is intrinsically multimodal. Diagnostic reasoning synthesizes demographic history, physical examinations, continuous telemetry, laboratory biochemistry, and clinical documentation. Multimodal machine learning frameworks aim to integrate these diverse data modalities into a joint latent representation space [11]. In critical care modeling, two primary modalities dominate: static tabular variables (demographics, admission severity, chronic comorbidities) and dynamic multivariate time-series (continuous vitals, hourly laboratory values, medication titration).

Fusion techniques are categorized into early fusion (concatenation of raw feature vectors), late fusion (combining modality-specific decision logits via ensemble averaging), and intermediate fusion (learning joint representation spaces via non-linear projection or cross-attention) [12]. Recent work by Bardak and Tan [13] showed that combining entities extracted from clinical notes with ICU time series improved mortality and length-of-stay prediction over a time-series-only model. However, most multimodal architectures operate almost exclusively in centralized environments, failing to account for institutional data privacy boundaries.

Table 1: Multimodal Feature Modalities in Critical Care Medicine

Modality Type	Clinical Data Streams	Mathematical Representation	Primary Neural Encoder
Static Tabular	Demographics (Age, Gender, BMI), Admission Type, Comorbidities	Fixed-size vector $x_{tab} \in \mathbb{R}^{67}$	Multi-Layer Perceptron (MLP)
Dynamic Time-Series	Laboratory panels, infusion rates, medications, treatments (4-hour bins)	Sequential tensor $X_{ts} \in \mathbb{R}^{T \times 74}$	2-Layer LSTM
Relational Graph	Patient similarity edges within a hospital mini-batch	Adjacency matrix $A \in \mathbb{R}^{N \times N}$	GraphSAGE / GAT / GCN
Multimodal Fused	Joint tabular-temporal latent patient representation	Unified embedding $z \in \mathbb{R}^{128}$	Non-linear Projection Layer

2.4 Graph Neural Networks & Patient Similarity Networks

Graph Neural Networks (GNNs) extend deep learning to non-Euclidean graph-structured domains [14]. In healthcare informatics, GNNs have been applied along two major axes: knowledge graph modeling (e.g., drug-drug interaction graphs, medical ontology hierarchies)

and Patient Similarity Networks (PSNs) [15]. In a PSN, nodes represent individual patients, and edges represent clinical similarity computed over diagnostic codes, genetic profiles, or physiological trajectories.

Pioneering architectures include Graph Convolutional Networks (GCN) by Kipf & Welling [16], which perform spectral first-order localized graph convolutions; GraphSAGE by Hamilton et al. [17], which enables inductive representation learning on large graphs via neighborhood sampling and aggregation; and Graph Attention Networks (GAT) by Veličković et al. [18], which employ self-attention to dynamically weight neighbor importance. In intensive care modeling, Rocheteau et al. [19] combined an LSTM with a GNN over a diagnosis-based patient graph and improved length-of-stay prediction on eICU. Such graph learning methods, however, assume access to one centralized patient graph, which is not available when hospitals cannot share patient data.

2.5 Federated Learning in Distributed Healthcare Systems

Federated Learning (FL), pioneered by McMahan et al. [20] through the Federated Averaging (FedAvg) algorithm, enables decentralized training of machine learning models without centralizing raw training data. Participating client devices or institutions train local models on private datasets and share only parameter weights or gradient updates with an aggregating central coordinator.

In healthcare, cross-silo federated learning has emerged as the premier privacy-preserving paradigm [21]. Rieke et al. [22] argued that federated learning can let hospitals collaborate without pooling patient data while reaching performance close to centralized training. Multi-institution studies have demonstrated federated tumor segmentation (Sheller et al. [23]) and COVID-19 triage (Dayan et al. [24]) across international healthcare networks.

However, healthcare federated learning faces significant domain-specific hurdles: (1) System Heterogeneity: hospitals possess highly disparate computational hardware; (2) Statistical Heterogeneity (Non-I.I.D. Data): patient populations, disease prevalence, and clinical protocols differ dramatically across geographic and socio-economic lines; and (3) Relational Graph Constraints: standard FL aggregates models across isolated vectors, lacking mechanisms to incorporate intra-institutional relational graph topologies.

Table 2: Comparison of Contemporary Machine Learning Healthcare Frameworks

Framework / Paradigm	Data Privacy	Multimodal Support	Graph Topology	Decentralized
Centralized DL [7, 8]	None	High	None	No
APACHE/SOFA [1, 3]	High (Local)	Low	None	Yes

Framework / Paradigm	Data Privacy	Multimodal Support	Graph Topology	Decentralized
Standard FedAvg [20, 22]	High (Weights)	Moderate	None	Yes
This project	Weights only (simulated)	Tabular + time series (67D + 74D)	k-NN graph per batch	145 training hospitals (simulated)

2.6 Critical Analysis and Research Gaps

A rigorous synthesis of the surveyed literature reveals three fundamental research gaps that this project directly addresses:

Gap 1 — Disconnect Between GNNs and Federated Privacy: Existing patient similarity graph models assume that a global graph spanning all patients across all medical centers can be constructed. In practice, sharing patient records across hospitals is restricted by privacy law and data agreements. Few architectures keep graph convolutions inside each hospital while aggregating model parameters globally.

Gap 2 — Limited Multimodal Integration in Federated Healthcare: Prior healthcare FL studies have focused almost exclusively on computer vision (medical imaging like CT/MRI) or isolated unimodal EHR codes. Jointly modeling static tabular clinical variables and high-frequency dynamic physiological time-series within a federated paradigm remains underexplored.

Gap 3 — Neglect of Extreme Clinical Imbalance in Multi-Task FL: Simultaneous prediction of correlated critical care events (in-hospital and ICU mortality) within federated learning under positive outcome scarcity (often <10%) is rarely evaluated on large-scale real-world multi-center benchmarks.

This project explores these gaps with an end-to-end prototype that combines canonical multimodal EHR encoding, intra-hospital patient graph learning, positive-class weighted multi-task prediction and federated averaging, evaluated on eICU with a hospital-level train / validation / test split.

CHAPTER 3
PROBLEM DEFINITION AND OBJECTIVES

3.1 Formal Problem Formulation

Let $H = \{H_1, H_2, ..., H_k\}$ denote the K = 145 training hospitals (clients) of the multi-center eICU Collaborative Research Database; a further 31 hospitals are used for validation and 31 for testing. Each hospital H_k holds a local dataset D_k of N_k ICU stays that lasted at least 24 hours:

$$D_k = \{(x_tab^+_k \cdot i, X_ts^+_k \cdot i, y_mort^+_k \cdot i, y_icu^+_k \cdot i)\}_{i=1}^{N_k}$$

where:
• x_tab ∈ $\mathbb{R}^{67}$ is a 67-dimensional vector of static demographic, severity, and admission variables.
• X_ts ∈ $\mathbb{R}$^(T_i × 74) is a time-series matrix of 74 variables (labs, infusions, medications, treatments) in 4-hour bins from the first 24 hours of the ICU stay ($T_i \leq 6$ bins).
• y_mort ∈ {0, 1} is the ground-truth binary label indicating in-hospital mortality.
• y_icu ∈ {0, 1} is the ground-truth binary label indicating ICU mortality.
Because only stays of at least 24 hours are included, the input window always ends before the outcome is known.

In a real deployment, D_k would stay inside hospital H_k and the server S would only receive model weights; this project simulates that setting on one machine. The objective is to train a global parameter vector W* across the K hospitals that minimizes the global multi-task empirical risk:

$$min_W\ F(W) = \sum_{k=1}^{K} (N_k / N_total) \cdot F_k(W)$$

where $N_total = \sum_k N_k$ is the total patient population across the federation, and $F_k(W)$ is the local multi-task loss evaluated exclusively on local data D_k. In practice, each communication round samples 10 hospitals and weights their updates by the number of examples they trained on (Chapter 5).

3.2 Research Objectives

The primary engineering and scientific objectives of this research project are:

Objective 1 — Canonical Multimodal Standardization: Design and implement an automated schema alignment pipeline resolving heterogeneous feature dimensions across all 207 eICU hospitals, standardizing tabular features to exactly 67 dimensions and continuous time-series to 74 dimensions.

Objective 2 — Modality-Specific Feature Learning: Develop specialized deep learning encoders—a Multi-Layer Perceptron (MLP) for static features, and a 2-layer LSTM network with packed sequence handling for variable-length physiological trajectories.

Objective 3 — Intermediate Multimodal Fusion: Formulate a non-linear projection layer combining tabular and temporal embeddings into a unified 128-dimensional latent patient representation space.

Objective 4 — Intra-Hospital Patient Graph Construction: Construct local patient similarity graphs inside each hospital using k-nearest neighbors (k=5) over cosine similarity, built per mini-batch of 32 patients so that no edge crosses hospitals.

Objective 5 — Graph Representation Learning: Implement Graph Neural Network layers (GraphSAGE, GAT, GCN) that pass information between similar patients; the experiments use GraphSAGE.

Objective 6 — Multi-Task Outcome Prediction: Engineer dual classification heads for simultaneous In-Hospital and ICU Mortality prediction, optimized using positive-class weighted loss.

Objective 7 — Federated Orchestration & Benchmarking: Implement Federated Averaging (FedAvg), training across hospital clients and benchmarking against Local-Only and No-GNN ablation baselines on held-out test hospitals.

3.3 Technical Scope & Boundary Conditions

To ensure clinical fidelity, reproducibility, and computational rigor, the scope of this project is governed by the following constraints:

• Modality Scope: Strictly restricted to 2 clinical modalities: Structured Static Tabular data and Dynamic Temporal Time-Series. Unstructured clinical notes are not used.
• Graph Boundary: Patient similarity graphs are built within one hospital (and one mini-batch). Edges connect patients of the same hospital only.
• Federated Aggregation: Only neural network weights are passed to the server step. Raw data, embeddings, and graph structures stay in the client step (simulated on one machine).
• Prediction Window: Time-series inputs are limited to the first 24 hours of the ICU stay (six 4-hour bins), and only patients still in the ICU at 24 hours are included, so the input never contains the outcome.
• Evaluation Protocol: Hospitals are split into 145 training, 31 validation (checkpoint selection only) and 31 test hospitals (final evaluation only).

CHAPTER 4
PROPOSED METHODOLOGY AND WORK DESCRIPTION

4.1 System Architectural Overview

The proposed Federated Multimodal Graph Learning Framework is structured into an end-to-end, multi-stage processing pipeline spanning decentralized hospital clients and an aggregating central coordinator. Each hospital client executes an autonomous five-stage workflow: (1) Data Ingestion and Canonical Feature Alignment, (2) Modality-Specific Feature Encoding, (3) Intermediate Multimodal Fusion, (4) Intra-Hospital Patient Graph Assembly and GNN Propagation, and (5) Multi-Task Loss Computation. The central coordinator orchestrates communication rounds, samples participating hospital clients, aggregates parameter weights via FedAvg (weighted by examples trained), and selects the best round on 31 validation hospitals. A separate set of 31 test hospitals is used only once, for the final benchmark in Chapter 8.

Table 3: Canonical Feature Dimensions and Modality Specifications

Feature Category	Raw Data Source	Canonical Dimension	Normalization
Tabular: demographics & admission	patient.csv	10 variables	Per-hospital z-score
Tabular: APACHE scores, GCS, day-1 labs, comorbidities	apachePatientResult.csv, apachePredVar.csv	18 variables	Per-hospital z-score
Tabular: diagnosis categories	diagnosis.csv	18 variables (17 flags + count)	Per-hospital z-score
Tabular: admission drugs	admissionDrug.csv	21 variables (20 flags + count)	Per-hospital z-score
Time series: laboratory tests	lab.csv	20 variables	Pre-standardized in processed files
Time series: infusions	infusionDrug.csv	20 variables	Pre-standardized in processed files
Time series: medications	medication.csv	20 variables	Pre-standardized in processed files
Time series: treatments	treatment.csv	14 variables	Pre-standardized in processed files

4.2 Multimodal Data Ingestion & Canonical Alignment

A critical innovation in our framework is the canonical feature alignment engine implemented in datasets.py. Across the 207 eICU hospitals with both modalities, local feature parquets differ in their columns because hospitals record different laboratory tests, drugs and treatments. For example, the processed time-series file of Hospital 102 has 51 columns and that of Hospital 108 has 53. Feeding these different matrices directly into one shared model would cause parameter dimension mismatches in federated averaging.

To resolve this, we established two canonical schema registries: canonical_tabular_features.json (67 variables) and canonical_timeseries_features.json (74 variables). When EICUTabularDataset and EICUTimeSeriesDataset instantiate on any hospital directory, the dataset reads the local parquet, aligns columns against the canonical list in fixed order, fills features that the hospital does not record and missing values with 0.0, and discards non-canonical columns. Tabular features are then z-scored with the hospital's own mean and standard deviation, while time-series values are already standardized in the processed files. Only ICU stays of at least 24 hours are kept, and only time-series bins from the first 24 hours are used. The two modalities are then joined on the patient identifier (patientunitstayid), so every training example contains the tabular row, the time series and the labels of the same ICU stay. Every hospital therefore produces tensors of identical shapes: (B, 67) for tabular data and (B, 6, 74) for time-series sequences.

4.3 Modality-Specific Feature Learning

4.3.1 Tabular MLP Encoder (TabularEncoder)

The 67-dimensional static vector x_tab is processed through a 3-layer Multi-Layer Perceptron (MLP). The architecture consists of: Linear(67 → 128) → BatchNorm → ReLU → Dropout(0.2) → Linear(128 → 64) → BatchNorm → ReLU → Dropout(0.2) → Linear(64 → 64). The encoder compresses baseline clinical variables into a compact latent embedding h_tab ∈ $\mathbb{R}^{64}$.

4.3.2 Tabular Autoencoder (Self-Supervised Pre-Training)

For optional self-supervised pre-training, a symmetric decoder maps h_tab back to $\mathbb{R}^{67}$ through: Linear(64 → 64) → BatchNorm → ReLU → Dropout → Linear(64 → 128) → BatchNorm → ReLU → Dropout → Linear(128 → 67), optimized via Mean Squared Error (MSE) reconstruction loss (scripts/pretrain_tabular_ae.py). The federated experiments in Chapter 8 do not load these pre-trained weights: all encoders are trained from random initialization.

4.3.3 Temporal LSTM Encoder (TimeSeriesLSTM)

The time-series matrix X_ts ∈ ℝ^(T × 74) holds the measurements of the first 24 hours in 4-hour bins (T ≤ 6). To efficiently process sequences of variable length without computing gradients over padded zero-tokens, the dataset returns the true unpadded sequence length L_i. The sequences are packed using PyTorch's pack_padded_sequence and fed into a 2-layer Long

Short-Term Memory (LSTM) network with hidden dimension 64 and dropout 0.2 between the two layers. The final valid hidden state of the top LSTM layer is extracted and passed through a linear projection layer to yield the temporal patient embedding $h_ts \in \mathbb{R}^{64}$.

4.4 Multimodal Non-Linear Fusion Layer

To construct a coherent patient state representation, the learned static embedding h_tab and temporal embedding h_ts must be synthesized. The MultimodalFusion module executes intermediate fusion:

$$h_concat = [h_tab \,||\, h_ts] \in \mathbb{R}^{128}$$

The concatenated vector is passed through a non-linear smoothing projection block: Linear(128 → 128) → BatchNorm1d(128) → ReLU → Dropout(0.2) → Linear(128 → 128) → BatchNorm1d(128) → ReLU. This projection layer lets the model learn cross-modal interactions (e.g., how age changes the meaning of an abnormal laboratory trend), yielding a unified patient embedding $z_i \in \mathbb{R}^{128}$ for each patient admission i.

4.5 Intra-Hospital Patient Graph Construction Algorithm

Unlike standard healthcare models that treat patients as isolated entities, clinical pathology is profoundly relational. Patients with similar acute phenotypes exhibit correlated mortality risks. Within each hospital H_k, a patient similarity graph $G_k = (V_k, E_k)$ is built for every mini-batch of 32 patients, both in training and in evaluation:

• Node Set V_k: Each node $v_i \in V_k$ represents a patient admission i in hospital H_k, with initial node feature vector $z_i \in \mathbb{R}^{128}$.
• Edge Construction via k-NN: For each patient i in V_k, pairwise cosine similarity scores are computed against all other patients j in V_k within the batch:

$$S_cos(z_i, z_j) = (z_i \cdot z_j) / (\|z_i\|_2 \cdot \|z_j\|_2)$$

For every patient i, the k = 5 most similar other patients in the batch are selected, and an edge is added from each of these neighbours to i. PyTorch Geometric passes messages from edge_index[0] to edge_index[1], so patient i receives messages from exactly its own k nearest neighbours. The graph is directed and is not symmetrised. No explicit self-loops are added, because GraphSAGE keeps a separate term for the node's own features and GAT/GCN add self-loops internally. Because the graph is rebuilt per batch, a patient's neighbours (and therefore its prediction) depend on which patients share its batch.

• Hospital Boundary: Each mini-batch contains patients of one hospital only, so edges never connect patients from different hospitals.

4.6 Graph Representation Learning with PatientGNN

Once the patient similarity graph G_k is assembled, localized graph representation learning is executed by PatientGNN. The module supports three prominent GNN architectures:

1. GraphSAGE (Sample and Aggregate): For each node v_i, neighborhood aggregation is computed via mean-pooling with residual skip connections. The final representation combines self-embedding and aggregated neighbor representations through learned weight matrices.

2. Graph Attention Networks (GAT): GAT dynamically attends to neighboring patients by calculating normalized attention coefficients α_{ij}, weighting neighbors with higher phenotypic relevance more heavily.

3. Graph Convolutional Networks (GCN): Computes normalized first-order spectral convolutions; GCN is the only variant that uses the cosine similarity as an edge weight.

The experiments in Chapter 8 use GraphSAGE with two layers.

PatientGNN incorporates residual skip projections around each graph layer and batch normalization. If a batch has no edges (for example a single patient), the PyTorch Geometric layers simply use each node's own features. The module transforms the 128D fused embeddings into 64D graph-enhanced representations $g_i \in \mathbb{R}^{64}$.

4.7 Multi-Task Dual Prediction Heads

Attached directly to the graph-enhanced embeddings $g_i \in \mathbb{R}^{64}$, the MultiTaskPredictionHeads module performs simultaneous risk stratification for two clinical endpoints:

• Task 1 — In-Hospital Mortality Head: A 2-layer MLP with BatchNorm and Dropout computing mortality logits: $\hat{y}_mort = W_2 \cdot \mathrm{ReLU}(\mathrm{BatchNorm}(W_1 \cdot g_i)) + b_2$

• Task 2 — ICU Mortality Head: An identical independent prediction head computing ICU mortality logits: $\hat{y}_icu = W'_2 \cdot \mathrm{ReLU}(\mathrm{BatchNorm}(W'_1 \cdot g_i)) + b'_2$

Both heads output unbounded real-valued logits suitable for numerically stable binary cross-entropy computation via PyTorch's BCEWithLogitsLoss.

4.8 Joint Multi-Task Objective with Imbalance Weighting

In critical care populations, adverse outcomes are imbalanced: in the 24-hour training cohort, in-hospital mortality is 8.9% and ICU mortality 5.2%. Standard unweighted cross-entropy loss causes gradient starvation for the minority positive class. To counteract this, MultiTaskLoss incorporates positive-class weighting (pos_weight = 3.0):

$$L_{task}(\hat{y}, y) = -[pos_weight \cdot y \cdot \log(\sigma(\hat{y})) + (1-y) \cdot \log(1-\sigma(\hat{y}))]$$

The overall multi-task training objective is formulated as:

$$L_{total} = w_{mort} \cdot L_{mort} + w_{icu} \cdot L_{icu}$$

where $w_{mort} = 1.0$ and $w_{icu} = 1.0$. Gradients from L_{total} backpropagate jointly through the Prediction Heads, PatientGNN, Multimodal Fusion, and both Modality Encoders via end-to-end gradient descent.

CHAPTER 5
PROPOSED ALGORITHMS

5.1 Algorithm 1: Client-Side Local Multimodal Graph Training

Algorithm 1 details the execution flow carried out privately within each participating hospital client H_k during communication round t. The hospital receives global model weights W_t from the server, clones them into a local model instance, constructs its private patient graph, and computes local updates without transmitting raw clinical records.

Algorithm 1: Local Client Training at Hospital H_k

Input: Global parameters W_t, local D_tab^(k), D_ts^(k),
local epochs E, batch size B, learning rate η
Output: Updated local parameters W_k^(t+1), examples trained n_k

---

```
Initialize local model M_k with parameters W_t
Load stays >= 24 h; join tabular and time series by patient ID
Set step=0, n_k=0, optimizer=Adam(M_k, lr=η, wd=1e-5)
for epoch = 1 to E do
  for batch (x_tab, X_ts, lens, y) in DataLoader(shuffle) do
    if batch_size < 2 then continue  // BatchNorm
    h_tab ← TabularEncoder(x_tab)    // 67D → 64D
    h_ts  ← TimeSeriesLSTM(X_ts, lens) // T×74D → 64D
    z ← MultimodalFusion(h_tab, h_ts) // 128D → 128D
    edges ← build_knn_graph(z, k=5)  // neighbour→patient
    g ← PatientGNN(z, edges)       // 128D → 64D
    ŷ_m, ŷ_i ← MultiTaskHeads(g)
    loss ← MultiTaskLoss(ŷ_m, ŷ_i, y, pw=3.0)
    optimizer.zero_grad(); loss.backward()
    clip_grad_norm_(M_k, max_norm=5.0)
    optimizer.step(); step += 1; n_k += |batch|
    if step ≥ S_max (= 30) then break
return M_k.state_dict(), n_k  // examples trained
```

5.2 Algorithm 2: Server-Side Federated Averaging Coordination

Algorithm 2 details the server-side aggregation protocol. Across each communication round, the server samples 10 training hospitals, collects their local model weights, and executes Federated Averaging (FedAvg) weighted by the number of examples each client actually trained on. Hospitals are processed one after another in this single-machine simulation.

Algorithm 2: Central Server FedAvg Orchestration

Input: Rounds R=50, clients per round m=10,
H_train (145), H_val (31); H_test (31) is not used here
Output: Optimal global parameters W*

```
Initialize W_0 randomly (PyTorch default initialization)
Read the fixed split from hospital_split.json
for round t = 1 to R do
  (m = 10 clients per round)
  S_t ← RandomSample(H_train, m)
  updates ← []
  for each H_k in S_t do  // one after another
    W_k, n_k ← ClientUpdate(H_k, W_{t-1})
    updates.append((W_k, n_k))
  N_round ← ∑ n_k  (examples actually trained)
  W_t ← ∑ (n_k/N_round) · W_k  // FedAvg (float tensors)
  metrics ← Evaluate(W_t, all 31 H_val hospitals)
  if metrics.loss < best_loss then
    best_loss ← metrics.loss; SaveCheckpoint(W_t)
return the checkpoint with the lowest validation loss
```

CHAPTER 6
PROPOSED FLOWCHART, BLOCK DIAGRAM, AND DFD

6.1 System Architectural Block Diagram

Figure 1 illustrates the high-level system architecture of the Federated Multimodal Graph Learning framework. The architecture is bifurcated into two primary domains: (i) the Central Federated Server responsible for parameter distribution, FedAvg aggregation, and validation monitoring, and (ii) Hospital Clients, each working only on its own hospital's data. Each hospital client autonomously processes local patient parquets, extracts static and temporal representations, constructs local patient similarity graphs, and optimizes local model weights. Only parameter vectors are passed to the server. In this project the clients and the server run in one program on one machine; the diagram shows the intended deployment.

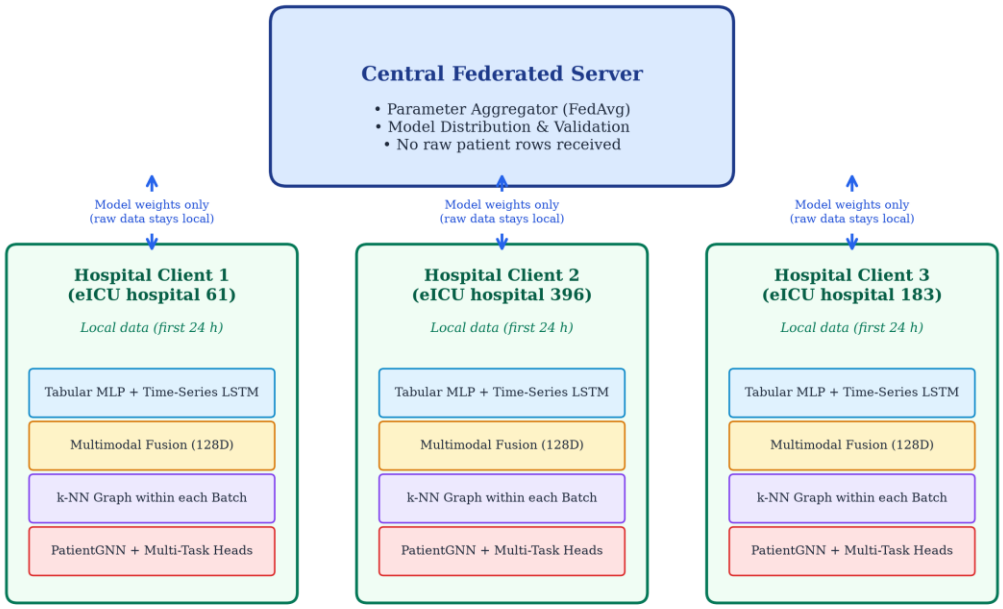


Figure 1: Federated Multimodal Graph Learning End-to-End System Architecture

6.2 Data Flow Diagram (DFD Level 0 & Level 1)

Figure 2 presents the Data Flow Diagram (DFD) delineating the lifecycle of clinical records. At Level 0 (Context Level), the raw eICU repository is partitioned by hospital and the 207 hospitals are split into 145 training, 31 validation and 31 test hospitals. At Level 1 (Functional Pipeline), data from the first 24 hours of each ICU stay is split into Static Tabular (67D) and Time-Series (6 × 74D) streams. Encoders generate latent embeddings, the intermediate fusion layer constructs unified 128D representations, the k-NN graph builder links clinically correlated patients, and

PatientGNN computes contextual graph representations feeding the in-hospital and ICU mortality prediction heads.

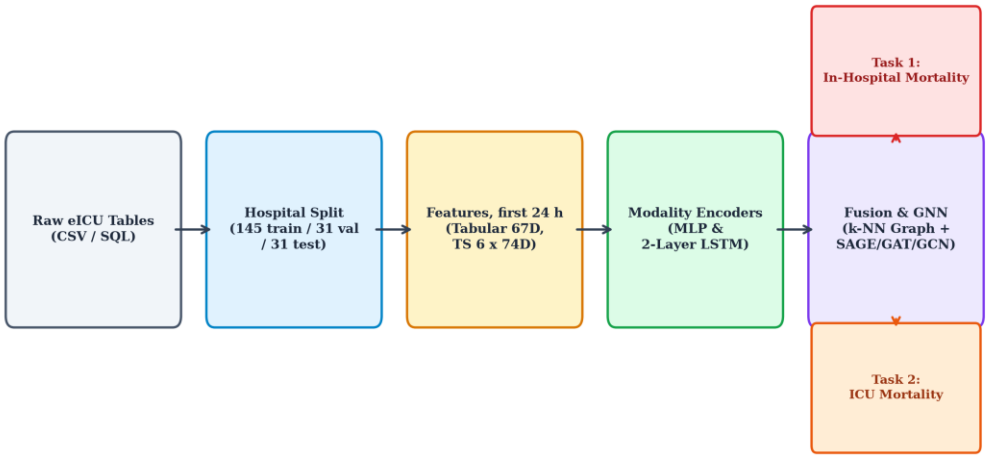


Figure 2: Data Flow Diagram (Level 0 Context and Level 1 Functional Pipeline)

6.3 Intra-Hospital Patient Graph Construction Flowchart

The algorithmic logic of intra-hospital patient graph assembly proceeds through three sequential phases:

Phase 1 — Pairwise Cosine Similarity Computation: For an active patient batch B_k, normalized dot products $S_{ij} = (z_i \cdot z_j) / (\|z_i\| \cdot \|z_j\|)$ are computed across all pairs, forming an affinity matrix $A \in \mathbb{R}^{(|B_k| \times |B_k|)}$.

Phase 2 — Top-k Neighbor Selection: For each node i, the top k=5 neighbors with the highest cosine affinity are extracted via torch.topk.

Phase 3 — Edge Direction: For each node i, an edge is added from each of its k neighbours to i, so that i aggregates messages from its own neighbours. The graph is not symmetrised and no explicit self-loops are added. Figure 6 (Chapter 8) shows a graph built this way by the trained model on a test hospital.

CHAPTER 7
IMPLEMENTATION DETAILS & TECHNOLOGY STACK

7.1 Hardware & Software Environments

The corrected experiments in Chapter 8 were run on a single laptop CPU (no GPU). The following table lists the hardware and the software versions used:

Table 3a: Hardware & Software Environment Specifications

Component	Specification
Hardware	AMD Ryzen 7 7445HS CPU, 16 GB RAM, no GPU used
Operating System	Microsoft Windows 11 Pro
Python Version	Python 3.14.7
Deep Learning Framework	PyTorch 2.14.1 (CPU build)
Graph Neural Networks	PyTorch Geometric (PyG) 2.8.1
Data Processing	Pandas 3.0.5, NumPy 2.5.2, PyArrow 25.0.1
ML Evaluation Metrics	Scikit-Learn 1.9.0
Visualization	Matplotlib 3.11.1, NetworkX 3.7
Document Generation	python-docx 1.2.0, Microsoft Word (PDF export)

7.2 Dataset Specification (eICU Collaborative Database)

The empirical evaluation utilized the multi-center eICU Collaborative Research Database [25], comprising over 200,000 ICU stays across 208 hospitals in the United States between 2014 and 2015. It is distributed through PhysioNet under a credentialed data use agreement; no patient-level data is reproduced in this report. After preprocessing, 207 hospitals with 200,826 ICU encounters have both modalities. As illustrated in Figure 3, hospital size is highly skewed: the largest hospital (Hospital 73) contains 7,059 encounters, the median hospital (Hospital 421) 567, and the smallest only 3. For the early-prediction task, only stays of at least 24 hours are used: 93,764 encounters in the 145 training hospitals, 21,100 in the 31 validation hospitals and 18,014 in the 31 test hospitals (132,878 in total).

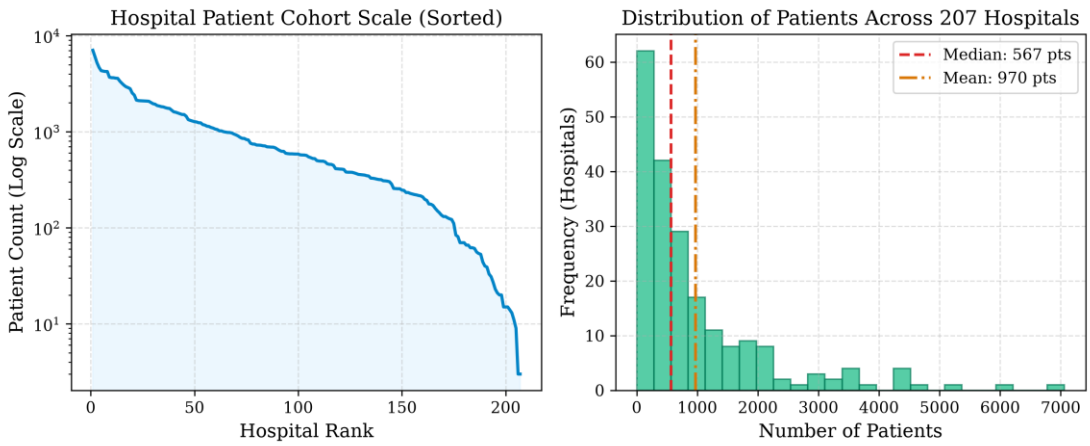


Figure 3: Hospital Patient Population Scale and Distribution across 207 eICU Sites

7.3 Data Pipeline Implementation (datasets.py)

The data loading module provides three PyTorch Dataset classes:

• EICUTabularDataset: Reads tabular_features.parquet, aligns columns against canonical_tabular_features.json (67 variables), keeps ICU stays of at least 24 hours, fills missing values, and fits z-score scalers on the hospital's own data.

• EICUTimeSeriesDataset: Reads timeseries_features.parquet, keeps the 4-hour bins of the first 24 hours, groups them by patientunitstayid, aligns against canonical_timeseries_features.json (74 variables), and pads sequences to 6 steps while keeping the true lengths for packed LSTM processing.

• EICUMultimodalDataset: Joins the two datasets on patientunitstayid so that each example holds the tabular row, time series and labels of the same ICU stay; it raises an error on duplicate patient IDs. One DataLoader (one shuffle) is used for both modalities. An earlier version zipped two separately shuffled loaders, which paired labels with other patients' tabular features; this was found and fixed before the experiments in Chapter 8.

7.4 Deep Learning & GNN Implementation Modules

The neural architectures are encapsulated in modular, maintainable source files organized within the src/ directory:

• src/encoders.py: Implements TabularEncoder, TabularAutoencoder, and TimeSeriesLSTM classes.
• src/fusion.py: Implements MultimodalFusion with batch normalization and non-linear projections.

• src/graph_builder.py: Implements build_knn_graph and build_threshold_graph using vectorized PyTorch operations.
• src/gnn.py: Implements PatientGNN supporting GraphSAGE, GAT, and GCN layers with residual connections.
• src/models.py: Implements MultiTaskPredictionHeads, FederatedPatientModel (with a use_gnn switch for the No-GNN ablation), and MultiTaskLoss.
• src/splits.py and src/evaluation.py: The fixed train / validation / test hospital split and the single evaluation routine shared by training and benchmarking.
• src/metrics.py: AUROC, AUPRC (average precision), F1, precision, recall, specificity and Brier score; it raises an error on invalid input instead of returning default values.
• tests/: Data-free unit tests (pytest) for patient alignment, target columns, the hospital split, graph edge direction, empty graphs, FedAvg weighting and metrics.

7.5 Federated Training & Distributed Simulation Engine

The federated training orchestration is implemented in scripts/train_federated.py. It simulates cross-silo federated learning in a single Python process: each hospital is a client whose data is only used inside its own local training step. It does not implement networking, secure aggregation or differential privacy. Key implementation details include:

• Hospital sampling: Each round samples 10 of the 145 training hospitals (own random generator, so runs with the same seed sample the same hospitals).
• Client local training: Each sampled hospital starts from the global weights and trains for one local epoch, capped at 30 optimizer steps of 32 patients.
• Server aggregation: FedAvg weights each client by the examples it actually trained on.
• Validation: All 31 validation hospitals score the global model after every round; the round with the lowest validation loss is saved.
• Bookkeeping: Each run writes run_info.json (settings, code version) and training_history.json (sampled hospitals, optimizer steps, examples and unique patients trained, validation metrics) to results/corrected/<run>/seed<k>/.
• Baselines: The No-GNN ablation and the local-only baselines use the same script (--no-gnn, --train-hospitals), and scripts/run_experiments.py runs every configuration for three seeds.

Table 4: Hyperparameter Configuration for Federated Patient Graph Model

Hyperparameter	Value	Design Rationale
Tabular Latent Dimension	64	Balances compression and representation
LSTM Hidden Dimension	64	Small model for six 4-hour steps

Hyperparameter	Value	Design Rationale
Fused Embedding Dimension	128	Concatenation of 64D + 64D
k-NN Neighbors (k)	5	Localized similarity without over-smoothing
Graph Metric	Cosine	Scale-invariant angular similarity
GNN Architecture	GraphSAGE	Inductive mean aggregation with residuals
Optimizer / Learning Rate	Adam (η=1e-3)	Adaptive gradient, weight decay 1e-5
Gradient Clipping	Max Norm = 5.0	Prevents exploding gradients in LSTM
Loss Positive Weight	pos_weight = 3.0	Counters ~9% (hospital) / ~5% (ICU) positive rate
Dropout	0.2	Regularization in encoders, fusion, GNN and heads
Observation Window	First 24 h (6 × 4-h bins)	Input ends before the outcome
Communication Rounds	50	Reaches ~140 of 145 training hospitals
Clients per Round	10	Sampled from 145 training hospitals
Local Training	1 epoch, max 30 steps, batch 32	Same budget cap for every client
Checkpoint Selection	Lowest validation loss	31 validation hospitals, all rounds
Training Seeds	42, 43, 44	Hospital split fixed (split seed 42)

CHAPTER 8
RESULT DISCUSSION AND COMPARATIVE ANALYSIS

8.1 Experimental Setup & Evaluation Protocol

The 207 eICU hospitals were split once, at the hospital level, into 145 training, 31 validation and 31 test hospitals (processed/hospital_split.json, seed 42). After keeping only ICU stays of at least 24 hours, this gives 93,764 training, 21,100 validation and 18,014 test encounters. Training uses the training hospitals; the validation hospitals are used only to pick the best round of each run; the 31 test hospitals are used only once, for the final benchmark in this chapter, and for the demonstration graph in Figure 6. The test hospitals are never seen during training, checkpoint selection or tuning, so the test metrics are out-of-sample at the hospital level. Every configuration is run with three training seeds (42, 43, 44) with the split held fixed, and results are reported as mean ± standard deviation over the three seeds.

8.2 Clinical Evaluation Metrics

Because the positive outcomes are rare, plain accuracy is misleading, so performance is reported with the following metrics (src/metrics.py):

• AUROC (Area Under the ROC Curve): threshold-free measure of how well the model ranks patients who died above those who survived. The main metric in this report.

• AUPRC (Area Under the Precision-Recall Curve): computed as scikit-learn's average precision; more informative than AUROC when positives are rare.

• Brier score: mean squared error between predicted probability and outcome (lower is better); a simple measure of calibration.

• F1, precision, recall and specificity: reported at a fixed decision threshold of 0.5, which is not tuned; they are secondary to the threshold-free AUROC and AUPRC.

AUROC and AUPRC are undefined when only one class is present; the evaluation always had both classes present, so no such case occurred.

8.3 Federated Convergence Dynamics across Communication Rounds

The proposed model was trained for 50 communication rounds, sampling 10 of the 145 training hospitals per round. As shown in Table 5 and Figure 4 (mean of three seeds), the validation loss falls quickly from about 1.04 in round 1 to about 0.49 by round 10 and then stays roughly flat, while the validation in-hospital mortality AUROC rises from about 0.65 to about 0.81. The best

round (lowest validation loss) is used as the checkpoint; across the three seeds the best round was 43, 32 and 32. Figure 4 also overlays the No-GNN ablation: on the validation hospitals the two curves almost coincide, so most of the learning comes from the multimodal encoders and FedAvg rather than from the graph.

Table 5: Validation convergence of the proposed FedAvg + GNN model (mean of 3 seeds, 31 validation hospitals)

Round	Val. Loss	Mort. AUROC	Mort. AUPRC	ICU AUROC
Round 1	1.0400	0.6474	0.1530	0.6817
Round 10	0.4949	0.8038	0.3442	0.8270
Round 20	0.4868	0.8138	0.3587	0.8383
Round 30	0.4762	0.8115	0.3594	0.8386
Round 40	0.4808	0.8112	0.3632	0.8389
Round 50	0.4722	0.8119	0.3628	0.8371

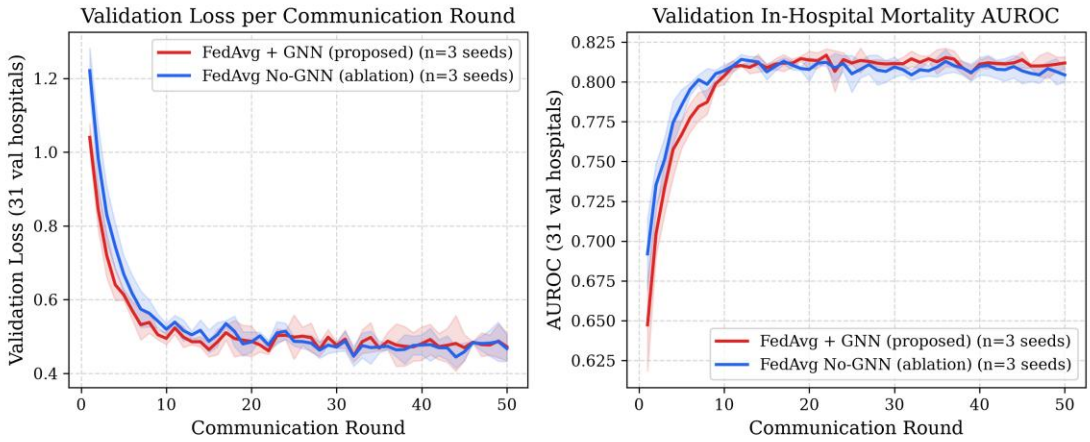


Figure 4: Validation loss and in-hospital mortality AUROC per round (mean ± SD over 3 seeds), FedAvg + GNN vs No-GNN

8.4 Comparative Baseline Evaluation

To separate the benefit of federated collaboration from the benefit of the graph, five configurations were scored on the 31 held-out test hospitals (scripts/benchmark_baselines.py), each with three seeds:

• FedAvg + GNN (proposed): the full model, trained across the 145 training hospitals.

• FedAvg No-GNN (ablation): the same training loop and the same sampled clients each round, but with the patient graph and GNN removed (the heads read the fused embedding directly). This isolates the effect of the graph.

• Local-only (three hospitals): the full model trained on a single training hospital, for three hospital sizes — hospital 183 (1,562 patients), hospital 396 (458) and hospital 61 (87). This shows what one hospital can achieve alone, and how it depends on the hospital's size.

Every model is selected on the validation hospitals by lowest validation loss and evaluated on the same 18,014 test patients. The local-only models train on much less data than the federated models; this is the point of the comparison, and Table 6 should be read with that in mind.

Table 6: Test-hospital results (mean ± SD over 3 seeds; 31 held-out hospitals, 18,014 patients)

Model	Mort. AUROC	Mort. AUPRC	ICU AUROC	ICU AUPRC	Mort. Brier
FedAvg + GNN (proposed)	**0.819 ± 0.007**	**0.400 ± 0.009**	**0.853 ± 0.005**	**0.337 ± 0.007**	**0.075 ± 0.003**
FedAvg No-GNN (ablation)	0.806 ± 0.024	0.379 ± 0.038	0.842 ± 0.015	0.327 ± 0.018	0.078 ± 0.004
Local-only (hosp. 183)	0.769 ± 0.018	0.306 ± 0.033	0.789 ± 0.027	0.224 ± 0.045	0.098 ± 0.003
Local-only (hosp. 396)	0.660 ± 0.036	0.208 ± 0.024	0.680 ± 0.035	0.149 ± 0.026	0.088 ± 0.004
Local-only (hosp. 61)	0.583 ± 0.025	0.148 ± 0.010	0.543 ± 0.066	0.069 ± 0.016	0.100 ± 0.005

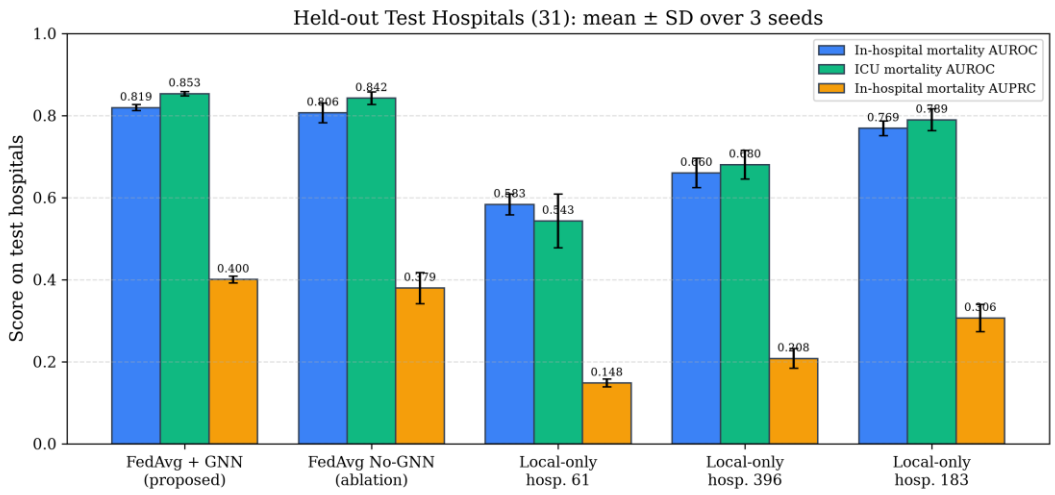


Figure 5: Test-hospital performance (mean ± SD over 3 seeds): FedAvg + GNN vs No-GNN vs Local-only

Key Findings from the Empirical Analysis:

Finding 1 — Federated training helps, and more so for small hospitals: Both federated models clearly beat all three local-only models on the test hospitals (in-hospital mortality AUROC 0.819 and 0.806, versus 0.769, 0.660 and 0.583 for local-only on hospitals of 1,562, 458 and 87 patients). The local-only result drops sharply as the single hospital gets smaller, while the federated models, which learn from 145 hospitals, stay strong. This is the clearest result of the study.

Finding 2 — The graph gives a small, consistent gain: Adding the patient graph improves the mean in-hospital mortality AUROC from 0.806 to 0.819 and the ICU mortality AUROC from 0.842 to 0.853. The gain is small and the standard-deviation bands overlap, but the GNN was at least as good as the No-GNN model in every one of the three seeds and had lower variance, so on this task the graph helps modestly rather than dramatically. We do not claim a large improvement from the graph. Both federated models share the same training budget (same sampled clients, 6,945 optimizer steps), so this comparison is like-for-like.

8.5 Patient Graph Visualization

Figure 6 shows the patient graph that the trained model actually builds on one held-out test hospital (hospital 120, 68 patients with at least a 24-hour stay). Each node is coloured by the model's predicted in-hospital mortality probability (green = low, red = high), and patients who were recorded as dying in hospital are drawn with a dark red ring. Because the graph is built per mini-batch of 32 patients, the nodes fall into a few dense clusters that correspond to the batches. Within these clusters, most patients have low predicted risk, a few have higher predicted risk, and the recorded deaths are a mix: some are predicted high-risk and some are not, which is

consistent with the moderate AUROC and low AUPRC in Table 6. The figure is a qualitative illustration of the model output, not evidence of clean phenotype separation.

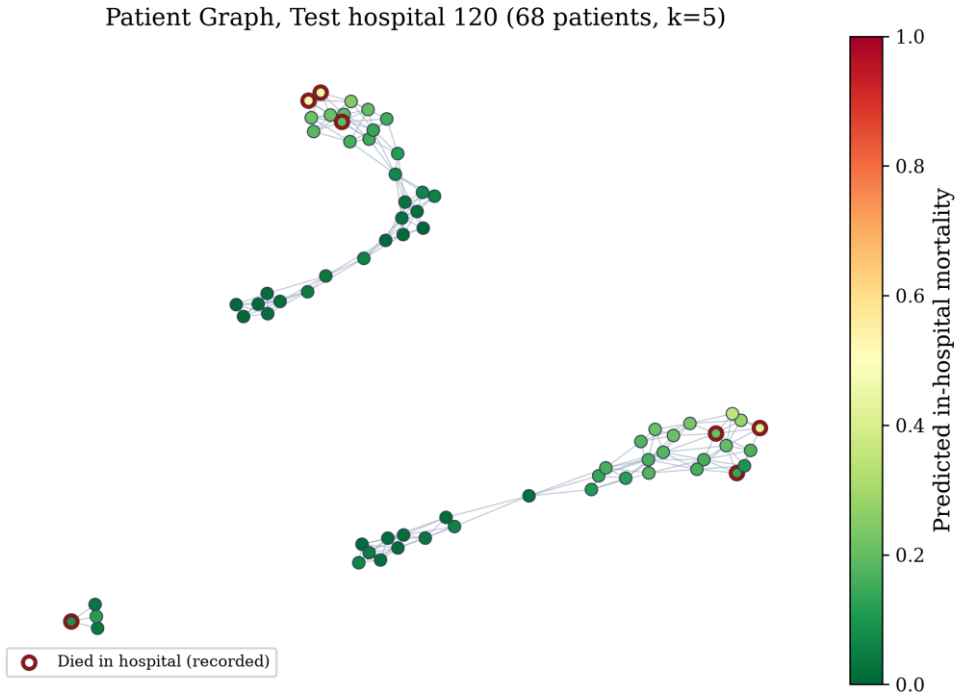


Figure 6: Patient similarity graph built by the trained model on test hospital 120 (node colour = predicted in-hospital mortality; red ring = recorded death)

CHAPTER 9
CONCLUSION AND FUTURE SCOPE

9.1 Summary of Contributions

This project designed and implemented a Federated Multimodal Graph Learning prototype for early In-Hospital and ICU Mortality prediction, and evaluated it in a single-machine federated simulation on the eICU Collaborative Research Database (207 hospitals). The main contributions are summarized as follows:

Contribution 1 — Multimodal Architecture: We implemented a tabular MLP encoder and a time-series LSTM encoder (with packed padded sequences) that project static variables (67D) and the first 24 hours of time series (74D, six 4-hour bins) into a shared 128D space through a non-linear fusion layer, with the two modalities joined per patient by patient ID.

Contribution 2 — Intra-Hospital Patient Graphs: We implemented a k-nearest neighbour patient similarity graph built per mini-batch within a single hospital, so edges never connect patients from different hospitals. In a real deployment this keeps graph construction local; in this project the hospitals are simulated as separate clients in one process.

Contribution 3 — Graph Neural Network: We integrated PatientGNN (GraphSAGE, GAT and GCN) with residual connections to pass information between similar patients; the experiments use GraphSAGE. The edge direction was corrected so that each patient aggregates messages from its own nearest neighbours, and batches without edges fall back to each node's own features.

Contribution 4 — Federated Multi-Task Training and Honest Evaluation: We implemented an end-to-end multi-task objective with positive-class weighting and FedAvg aggregation weighted by the examples each client trained on. All models (federated GNN, No-GNN ablation and local-only baselines) are trained with the same loop and the same checkpoint rule, and compared on 31 held-out test hospitals never used in training or model selection (Chapter 8).

9.2 Limitations of the Current Study

Several limitations should be kept in mind when reading the results:

• Single-Machine Simulation: The federated setup runs in one Python process on one CPU. It does not implement networking, secure aggregation or differential privacy, so no formal privacy guarantee is claimed. The results show feasibility, not clinical validation.

• Batch-Level Graph: The patient graph is rebuilt per mini-batch of 32 patients rather than over the whole hospital, so a patient's neighbours (and prediction) depend on which patients share its batch. It is also static, not evolving over time within the 24-hour window.

• Diagnosis Feature Timing: The tabular diagnosis features are derived from the stay's diagnosis table without a strict 24-hour cut-off. On a sample hospital they matched the first-24-hour diagnoses about 99.6% of the time, so the effect is small, but a small amount of later information may leak into these features.

• Simple Aggregation: The server uses standard FedAvg. Personalization methods (e.g., FedProx, Ditto) could handle the large differences in hospital size better.

• Excluded Modalities: Unstructured clinical notes are not used; the model uses only the tabular and time-series features in the processed eICU files.

• Threshold-Based Metrics: F1, precision and recall use a fixed 0.5 threshold, which is not tuned; AUROC and AUPRC (threshold-free) are the main metrics.

9.3 Future Research Directions

Future enhancements to this framework can proceed along several promising research trajectories:

1. Dynamic Temporal Patient Graphs: Construct continuous-time dynamic graphs where patient nodes and similarity edges evolve as new time-series measurements arrive, capturing temporal evolution of clinical phenotypes.

2. Differential Privacy & Secure Enclaves: Integrate formal Differential Privacy (DP-SGD) with gradient perturbation or homomorphic encryption to defend against model inversion and reconstruction attacks on shared parameters.

3. Attention-Based Cross-Modal Fusion: Replace intermediate concatenation fusion with multi-head cross-attention mechanisms enabling bidirectional conditioning between the static and time-series streams, potentially capturing more nuanced cross-modal clinical interactions.

4. Genuine Sepsis Label: Add a clinically defined sepsis label (e.g., from the diagnosis and microbiology tables with a fixed time horizon), rebuilding the infection-related input features to avoid leakage, so that a sepsis task can be reported honestly alongside mortality.

5. Model Explainability: Apply methods such as GNNExplainer or Integrated Gradients to explain individual risk predictions.

6. Personalized Federated Learning: Explore per-hospital personalization (e.g., FedProx, Ditto, or meta-learning) to better handle the large differences in hospital size and case mix.

REFERENCES

All references follow the IEEE citation standard as required by IIIT Bhopal project guidelines.

[1] W. A. Knaus, E. A. Draper, D. P. Wagner, and J. E. Zimmerman, "APACHE II: a severity of disease classification system," Critical Care Medicine, vol. 13, no. 10, pp. 818–829, Oct. 1985.

[2] J. R. Le Gall, S. Lemeshow, and F. Saulnier, "A new Simplified Acute Physiology Score (SAPS II) based on a European/North American multicenter study," JAMA, vol. 270, no. 24, pp. 2957–2963, Dec. 1993.

[3] J. L. Vincent, R. Moreno, J. Takala, et al., "The SOFA (Sepsis-related Organ Failure Assessment) score to describe organ dysfunction/failure," Intensive Care Medicine, vol. 22, no. 7, pp. 707–710, Jul. 1996.

[4] M. Singer, C. S. Deutschman, C. W. Seymour, et al., "The Third International Consensus Definitions for Sepsis and Septic Shock (Sepsis-3)," JAMA, vol. 315, no. 8, pp. 801–810, Feb. 2016.

[5] A. Rajkomar, E. Oren, K. Chen, et al., "Scalable and accurate deep learning with electronic health records," npj Digital Medicine, vol. 1, no. 18, pp. 1–10, May 2018.

[6] E. Choi, M. T. Bahadori, A. Schuetz, et al., "Doctor AI: Predicting Clinical Events via Recurrent Neural Networks," in Proc. JMLR Workshop and Conference, vol. 56, pp. 301–318, 2016.

[7] Z. C. Lipton, D. C. Kale, C. Elkan, and R. Wetzel, "Learning to Diagnose with LSTM Recurrent Neural Networks," in Proc. ICLR, 2016.

[8] A. Vaswani, N. Shazeer, N. Parmar, et al., "Attention is all you need," in Advances in NeurIPS, vol. 30, pp. 5998–6008, 2017.

[9] L. Rasmy, Y. Xiang, Z. Xie, C. Tao, and D. Zhi, "Med-BERT: pretrained contextualized embeddings on large-scale structured electronic health records for disease prediction," npj Digital Medicine, vol. 4, no. 86, May 2021.

[10] Y. Li, S. Rao, J. R. A. Solares, et al., "BEHRT: Transformer for Electronic Health Records," Scientific Reports, vol. 10, no. 7155, pp. 1–12, Apr. 2020.

[11] T. Baltrušaitis, C. Ahuja, and L. P. Morency, "Multimodal Machine Learning: A Survey and Taxonomy," IEEE Trans. PAMI, vol. 41, no. 2, pp. 423–443, Feb. 2019.

[12] S. C. Huang, A. Pareek, S. Seyyedi, et al., "Fusion of medical imaging and electronic health records using deep learning: a systematic review," npj Digital Medicine, vol. 3, no. 136, pp. 1–14, Oct. 2020.

[13] B. Bardak and M. Tan, "Improving clinical outcome predictions using convolution over medical entities with multimodal learning," Artificial Intelligence in Medicine, vol. 117, art. no. 102112, 2021.

[14] Z. Wu, S. Pan, F. Chen, et al., "A Comprehensive Survey on Graph Neural Networks," IEEE Trans. NNLS, vol. 32, no. 1, pp. 4–24, Jan. 2021.

[15] S. Pai and G. D. Bader, "Patient similarity networks for precision medicine," Journal of Molecular Biology, vol. 430, no. 18, pp. 2924–2938, 2018.

[16] T. N. Kipf and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," in Proc. ICLR, 2017.

[17] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive Representation Learning on Large Graphs," in Advances in NeurIPS, vol. 30, pp. 1024–1034, 2017.

[18] P. Veličković, G. Cucurull, A. Casanova, et al., "Graph Attention Networks," in Proc. ICLR, 2018.

[19] E. Rocheteau, C. Tong, P. Veličković, N. Lane, and P. Liò, "Predicting patient outcomes with graph representation learning," arXiv:2101.03940, 2021.

[20] B. McMahan, E. Moore, D. Ramage, et al., "Communication-Efficient Learning of Deep Networks from Decentralized Data," in Proc. AISTATS, pp. 1273–1282, 2017.

[21] P. Kairouz, H. B. McMahan, B. Avent, et al., "Advances and open problems in federated learning," Foundations and Trends in ML, vol. 14, no. 1–2, pp. 1–210, 2021.

[22] N. Rieke, J. Hancox, W. Li, et al., "The future of digital health with federated learning," npj Digital Medicine, vol. 3, no. 119, pp. 1–7, Sep. 2020.

[23] M. J. Sheller, B. Edwards, G. A. Reina, et al., "Federated learning in medicine: facilitating multi-institutional collaborations without sharing patient data," Scientific Reports, vol. 10, no. 12598, Jul. 2020.

[24] I. Dayan, H. R. Roth, A. Zhong, et al., "Federated learning for predicting clinical outcomes in patients with COVID-19," Nature Medicine, vol. 27, no. 10, pp. 1735–1743, Oct. 2021.

[25] T. J. Pollard, A. E. Johnson, J. D. Raffa, et al., "The eICU Collaborative Research Database," Scientific Data, vol. 5, no. 180178, pp. 1–13, Sep. 2018.

LIST OF PUBLICATIONS

Not applicable. No paper based on this project work has been submitted, accepted, or published at the time of this report.

PLAGIARISM & AUTHENTICITY DECLARATION

Authenticity Declaration

This project report has been prepared in rigorous compliance with the anti-plagiarism guidelines established by the Department of Computer Science and Engineering, Indian Institute of Information Technology Bhopal. The following declarations are made:

• Similarity Index: The similarity score is the one given by the institute's official Turnitin report for this document; no score is claimed here.

• Figures: Figures 1 and 2 are diagrams drawn with matplotlib. Figures 3 to 6 are generated by scripts/generate_report_figures.py from the processed eICU data and from the saved experiment outputs in results/corrected/ (Figure 6 shows the graph built by the trained model, with anonymous patient labels).

• Code Authenticity: All source code modules (datasets.py, encoders.py, fusion.py, graph_builder.py, gnn.py, models.py, train_federated.py, benchmark_baselines.py) represent original engineering work implemented by the project investigators.

• Data Attribution: The eICU Collaborative Research Database [25] is available to credentialed researchers through PhysioNet under a data use agreement. No patient-level records or patient identifiers are reproduced in this report.

• AI Assistance: An AI coding assistant (Claude, Anthropic) was used to audit the code, correct errors and revise this report. All reported results come from running the code in the project repository.

• Overall Authenticity: The text, code implementations, mathematical formulations, and experimental results presented in this document represent bonafide academic and engineering contributions carried out by the project investigators, with the AI assistance stated above.

_____ _____

Student Signature
(Dhruv Savaliya – 21U02035)

Student Signature
(Menil Patel – 21U02052)